

Ton notebook ne sert à rien. (en prod)

...jusqu'à ce que quelqu'un puisse l'utiliser **sans toi.**

Du notebook à la production: à quoi ressemble un vrai job en ML.

~90% des modèles ne tournent **jamais** en production.

Sur Kaggle

Le modèle **EST** le produit.
Tu optimises un score, tu soumetts,
fini.

Dans la vraie vie

Le modèle = **~5% du boulot**.
Le reste : le faire tourner pour les
autres.

Le vrai job

Que ça marche **H24**, **sans** toi,
sur des données que tu n'as **jamais**
vues.

NOTEBOOK

✗ Tourne sur ta machine

✗ Toi seul l'exécutes

✗ `df.head()` pour vérifier

✗ "ça marche chez moi"

✗ Objectif : accuracy 0.94

PRODUCTION

✓ Tourne H24 sur un serveur

✓ 1000 requêtes / seconde

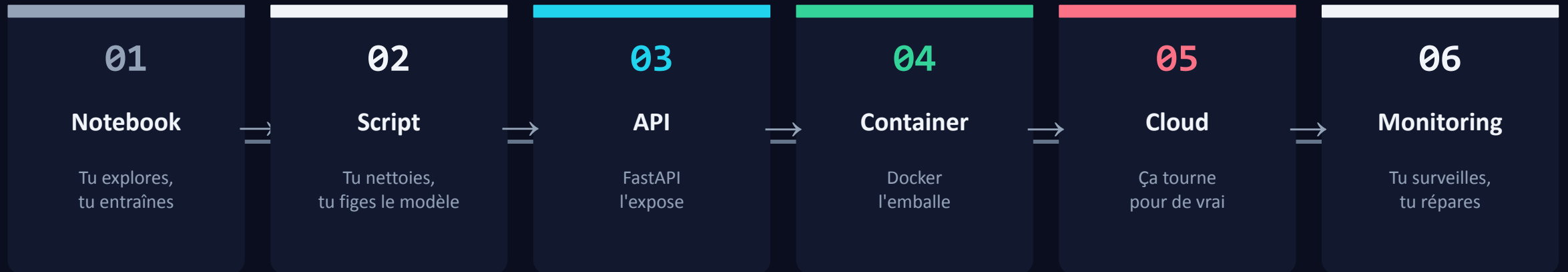
✓ Tests automatiques

✓ "ça marche partout"

✓ Objectif : < 200 ms, 99.9% up

LE VOYAGE DE TON MODÈLE

Du **.ipynb** à un truc que le monde appelle.



Aujourd'hui on s'arrête sur les 3 du milieu : **API · Container · Tracking.**

LES OUTILS QUE TU VAS VRAIMENT UTILISER

Pas de la théorie. Le kit de base de tout ingénieur ML.

FastAPI

Transforme ton modèle
en API en ~15 lignes.

Docker

"Ça marche chez moi"
→ ça marche partout.

MLflow

Quel modèle tourne ?
Avec quoi ?
Quelles métriques ?

Git + CI

Déployer sans tout casser.

Ton modèle devient une **API que le monde appelle.**

```
api.py

from fastapi import FastAPI
import joblib

app = FastAPI()
model = joblib.load("model.pkl")

@app.post("/predict")
def predict(features: list):
    pred = model.predict([features])
    return {"prediction": int(pred[0])}
```

Ce que ça change

- ▶ 15 lignes
- ▶ Une URL : POST /predict
- ▶ N'importe qui, n'importe quel langage, peut l'appeler.
- ▶ Ton modèle a quitté le notebook.

Fini le "ça marche chez moi".

bash

```
$ docker build -t mon-modele .
```

=> image construite ✓

```
$ docker run -p 8000:8000 mon-modele
```

=> API en ligne sur :8000 ✓

Un container =

- ▶ Ton code
- ▶ + ses dépendances
- ▶ + sa version de Python
- ▶ + son OS

Deux commandes. Ça tourne pareil sur ton Mac, sur le serveur, sur le cloud.

Quel modèle tourne en prod ? Avec quoi ? Pourquoi ?

Sans tracking : tu devines.

Avec MLflow : tu connais le passé et le présent.

MLflow · Experiments

RUN	MODÈLE	ACCURACY	DATE	STATUT
a1f3	rf_v3	0.942	06 juin	● live
9c2e	rf_v2	0.931	05 juin	archivé
4b7a	xgb_v1	0.928	04 juin	archivé
2d10	rf_v1	0.910	02 juin	archivé

Ce qu'une boîte attend de toi, ce n'est PAS le meilleur score Kaggle.

Du code lisible

Un autre humain doit pouvoir le reprendre sans toi.

Un modèle robuste

Qui ne casse pas à 3h du matin sur des données bizarres.

Savoir débbugger

Quand la prédiction part en vrille, tu trouves pourquoi.

Expliquer tes choix

Pourquoi ce modèle, ces données, ce seuil. En clair.

Un défi concret. 3 étapes. Ce week-end.

1

Prends UN modèle

N'importe lequel, même un vieux notebook Kaggle.

2

Sers-le avec FastAPI

Une route `/predict`. Teste-la dans `/docs`.

3

Mets-le dans Docker + pousse sur GitHub

Un Dockerfile, deux commandes, un repo public.

Tu viens de faire ce que **90% des juniors ne savent pas faire.**

Arrête de collectionner des notebooks. Commence à **livrer** des modèles.

Des questions ?

Repo + slides + démos → github.com/adnaneaabbar/notebook-to-prod